

RRTO: A High-Performance Transparent Offloading System for Model Inference in Mobile Edge Computing

Zekai Sun^{1,2}, Xiuxian Guan¹, Junming Wang¹, Yuhao Qing¹, Haoze Song¹,
Dong Huang¹, Fangming Liu^{3,4}, Heming Cui^{1,2,*}

Abstract—Machine Learning (ML) has become foundational for modern mobile applications. Deploying ML inference on resource-constrained mobile devices remains challenging due to limited computational resources and poor platform compatibility. While Mobile Edge Computing (MEC) offers offloading-based inference paradigm using GPU servers, existing approaches are divided into non-transparent and transparent methods, with the latter necessitating modifications to the source code. Non-transparent offloading achieves high performance but requires intrusive code modification, limiting compatibility with diverse or closed-source applications. Transparent offloading, in contrast, offers wide compatibility but introduces significant transmission delays due to per-operator remote procedure calls (RPCs). To overcome this limitation, we propose RRTO, the first high-performance transparent offloading system tailored for MEC inference. RRTO introduces a record/replay mechanism that leverages the static operator sequence in ML models to eliminate repetitive RPCs. To reliably identify this sequence, RRTO integrates a novel Operator Sequence Search algorithm that detects repeated patterns, filters initialization noise, and accelerates matching via a three-level strategy. Evaluation shows that RRTO reduces inference time and energy consumption by up to 50% and 75%, respectively, compared to state-of-the-art baselines, while requiring no source code modification.

Index Terms—Computation Offloading, Model inference, Robotic IoT, Distributed system and network

I. INTRODUCTION

Machine learning (ML) has become fundamental to a wide range of mobile applications, ranging from intelligent wearable sensors [1], [2] and autonomous vehicles [3], [4] to smart manufacturing systems [5], [6]. These applications depend on core technologies such as object detection [7]–[9], robotic control [10]–[12], and environmental perception [13]–[15], all of which require high-performance (low-latency and energy-efficient) inference. Deploying ML models on real-world mobile devices (e.g., smartphones, robots, IoT devices) faces two main challenges: platform compatibility, which requires supporting diverse software frameworks (e.g., PyTorch [16], TensorFlow [17], CUDA [18], OpenCL [19]) and hardware accelerators on mobile devices (e.g., GPU [20], FPGA [21], SoC [22]); and limited on-device computing resources, including restricted computational power and battery life.

Recent studies [1] have proposed mobile edge computing (MEC) as a promising solution for high-performance inference by leveraging GPU servers (e.g., edge devices with powerful GPUs) at the edge of the radio access network. Conventional MEC approaches (illustrated in Tab. I) fall into three categories: device-only inference, non-transparent offloading, and transparent offloading, categorized based on whether source code modification is required for enabling offloading. Device-only inference demands significant engineering effort due to poor platform compatibility and cannot deliver high performance because of limited on-device resources (Sec. II-A). As a result, many ML applications are turning to offloading-based inference over MEC networks, which offers high performance and broad compatibility by leveraging GPU servers.

Non-transparent offloading runs model inference on GPU servers but requires code modification, increasing engineering effort and limiting application diversity (i.e., support various upper-layer applications). A typical example is native offloading, which hosts the entire model remotely and improves performance (3.7× faster inference and 49% lower energy consumption compared to device-only inference). However, its non-transparency, caused by the need to access and modify source code, prevents application diversity with commercial closed-source software (e.g., TensorRT [23], CUDA-X library [24]) and runtime-optimized model structures (e.g., JIT [25]). Alternatives such as CUDA Graph [26] and TorchScript [27] reduce code modification but remain non-transparent and lack platform compatibility, as they support only specific frameworks (Sec. II-B).

Transparent offloading methods, such as Cricket [28], enable model inference to be offloaded to GPU servers without code modification, offering high convenience at the cost of reduced performance. During inference, ML models execute a sequence of operators (e.g., `torch.nn.functional.add()`, `torch.nn.functional.conv2d()` in PyTorch [16]), which are normally forwarded to backend system functions (e.g., `aten::add`, `aten::conv2d` within CUDA's `cudaLaunchKernel` [18]). Transparent offloading intercepts these calls and redirects their execution to GPU servers via Remote Procedure Calls (RPC), avoiding source code changes (Sec. II-C). However, since each operator triggers a separate RPC, the accumulated Round-Trip Time (RTT) introduces significant transmission overhead in MEC networks. This is because models often contain hundreds of operators (e.g., 522 in [9]) and invoke thousands of RPCs during inference (e.g., 5,895 in [9]); in wireless

*Corresponding author.

¹The University of Hong Kong, Hong Kong SAR, China. ²Shanghai AI Laboratory. ³Peng Cheng Laboratory. ⁴Huazhong University of Science and Technology. zksun@cs.hku.hk

Method	Category	Code Modification	Application Diversity	Platform Compatibility	High Performance
Device-only Inference	Device-only	N/A	✓	✗	✗
Native Offloading	Non-transparent	High	✗	✓	✓
CUDA Graph	Non-transparent	Low	✗	✗	✓
Cricket	Transparent	N/A	✓	✓	✗
RRTO (Ours)	Transparent	N/A	✓	✓	✓

TABLE I: Comparison of representative inference methods in MEC environments.

networks commonly used for mobile devices, each RTT can take several milliseconds [29], making transmission delay dominate inference time (up to 95% for Cricket [28]).

To address these challenges, one intuitive idea is to reduce transmission delay in transparent offloading by shifting from a reactive to a preactive approach. Existing transparent offloading methods trigger RPCs only after operators are invoked during inference, leading to sequential scheduling and communication delays. This reactive behavior stems from their general-purpose design for remote GPU usage, where operator patterns in upper-layer applications are often unpredictable, making proactive scheduling infeasible. In contrast, ML inference typically follows a fixed and predictable operator sequence due to the static computation graph (Sec. IV-A). This enables a preactive offloading mechanism that uses operator tracing and record/replay to anticipate operator sequences. By directly replaying recorded operators on the GPU server, the preactive approach eliminates per-operator RPC communication during the offloading process, significantly reduces transmission delay, and brings the performance benefits of non-transparent offloading back to transparent offloading.

However, integrating the record/replay mechanism into transparent offloading systems presents three key challenges in accurately identifying the correct operator sequence for each inference. First, transparent offloading systems intercept function calls from upper-layer applications to GPU devices at the system layer, where RRTO can only access low-level log records of the operators called without any direct hints from upper-layer applications, making it difficult to determine which inference task each operator belongs to. Second, operator RPCs generated during model loading and initial inference may differ from those in the regular inference loop, introducing variability that complicates sequence identification, where even small mismatches can affect correctness. Third, large operator traces, often containing tens of thousands of entries, create a vast search space that hinders timely identification of the correct sequence.

To tackle the above challenges, we propose **RRTO**, a **T**ransparent **O**ffloading system optimized for model inference in MEC with a novel **R**ecord/**R**eplay mechanism: RRTO automatically records the operators invoked during the first inferences using traditional RPCs as in traditional transparent offloading systems and replays the identified fixed-order operator sequence in subsequent inferences once the specific operator sequence for each inference is identified. To accurately identify this sequence, RRTO introduces the *Operator Sequence Search* algorithm. First, it leverages the observation that the target sequence is repeatedly embedded in the complete log during multiple inferences, ensuring task-level association of each

operator. Second, it verifies completeness by checking whether the repeated sequence aligns with the entire log, while ignoring inconsistencies caused by model loading or initialization. Third, a three-level fast match algorithm is applied in Operator Sequence Search to efficiently reduce the search space and accelerate sequence identification. The key contributions of this paper are summarized as follows:

- To the best of our knowledge, RRTO is the first high-performance transparent offloading system tailored for model inference in MEC. The code is released on <https://github.com/hku-systems/RRTO>.
- We propose a record/replay mechanism for transparent offloading system that eliminates per-operator RPC communication during offloading, significantly reducing transmission delay.
- We design a Operator Sequence Search algorithm that accurately reconstructs operator sequences to support the record/replay mechanism.
- we conduct extensive experiments demonstrating that RRTO outperforms state-of-the-art baselines in MEC environments, reducing inference latency by up to 50% and per-inference energy consumption by up to 75%, while achieving performance comparable to non-transparent offloading without requiring any source code modifications.

The rest of the paper is organized as follows. Sec. II motivates the design of RRTO by revealing the challenges in current MEC networks. Sec. III presents the overview of RRTO, with detailed design in Sec. IV. Sec.V introduces the system implementation, followed by performance evaluation in Sec.VI. Related works and technical limitations are discussed in Sec. VII. Finally, conclusions are presented in Sec.VIII.

II. BACKGROUND

A. Device-only Inference

Device-only inference, which runs models directly on mobile devices, faces two major limitations: poor platform compatibility and restricted performance. Mobile applications are built on diverse software frameworks (e.g., PyTorch [16], TensorFlow [17], CUDA [18], OpenCL [19]) and mobile devices are equipped with various hardware accelerators (e.g., GPU [20], FPGA [21], SoC [22]), making deployment on real-world devices labor-intensive and error-prone. Engineers must adapt each model to specific hardware and software configurations, sacrificing portability and increasing development cost. In addition, the performance of device-only inference is severely constrained by the limited computational power of processors and battery capacity. As shown in Fig. 1a, the inference latency on these devices exceeds the 30ms threshold required for smooth video fluency [30], [31] (indicated by the

red dotted line). Fig. 1b further shows that frequent inference reduces device standby time to 20–40% of its normal duration, significantly degrading user experience and device usability.

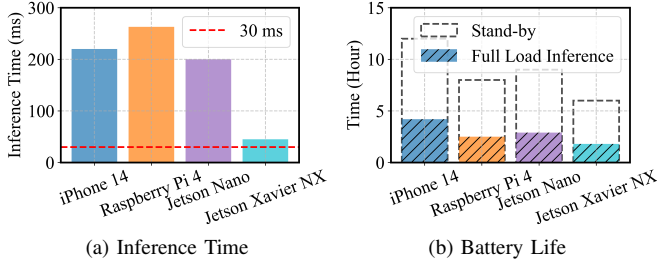


Fig. 1: The performance of VGG-16 under device-only inference across different mobile devices [32]–[37].

B. Non-Transparent Offloading

Non-transparent offloading executes model inference on GPU servers by transmitting only the raw input and output, but it requires modifications to the source code. This increases engineering complexity and limits its applicability to a wider range of upper-layer applications, particularly closed-source or runtime-adaptive ones. Although such methods can significantly improve performance (3.7× faster inference and 49% lower energy consumption compared to device-only inference), their need for application code access and modification makes them inherently non-transparent and restricts application diversity.

This non-transparency severely limits applicability in two key scenarios: commercial closed-source software and runtime-adaptive model structures. High-performance libraries such as TensorRT [23] and CUDA-X library [24] are widely adopted in mobile applications but do not expose source code, making them incompatible with non-transparent approaches. Many real-world applications also employ runtime optimization techniques, including just-in-time (JIT) compilation [25] and dynamic conventional kernel selection with different number/size based on task-specific requirements [38], [39]. These adaptive methods change model structure at runtime to improve speed and accuracy. Non-transparent offloading cannot accommodate such closed-source or dynamic environments. In contrast, transparent offloading intercepts system calls during runtime, enabling support for both closed-source libraries and runtime-adaptive model structures without requiring code modification.

Alternative methods such as CUDA Graph [26] and TorchScript [27] reduce the amount of code modification by packaging models for GPU server deployment. However, these approaches are still non-transparent and cannot support applications involving runtime variability or proprietary libraries. Further, they reduce platform compatibility by depending on specific software frameworks, which imposes greater constraints on mobile software development.

Various scheduling strategies have also been introduced in non-transparent offloading systems to improve performance while addressing goals such as latency and energy savings. Examples include layer partitioning [40]–[42] and multiple inference scheduling [43]–[48]. These optimizations provide

measurable benefits and can also be integrated into RRTO to further enhance performance (see Sec. VII for details).

Adapting existing optimizations from non-transparent offloading systems to RRTO. Advanced scheduling optimizations effective in non-transparent offloading systems are adaptable to RRTO.

Additionally, the advanced scheduling optimizations that have been proven effective in non-transparent offloading systems can be adapted to RRTO, and we leave it as a future work.

C. Workflow of Transparent Offloading

When a mobile application utilizes the GPU on the mobile device for inference (device-only inference), the complete system call flow from top to bottom is as follows (also illustrated in the left part of Fig. 2):

- A The ML application sequentially invokes the corresponding operators based on the ML model's structure to complete the entire computation process.
- B Each operator accesses the appropriate function library through a unified operator API tailored to the type of the device running the application; for instance, on NVIDIA GPUs, this is the CUDA runtime library ([18]).
- C The operating system loads the local CUDA runtime library by default, and executes the relevant CUDA kernel functions on the mobile device's GPU.
- D The local CUDA runtime library returns the computation results to the upper-layer application.

Transparent offloading methods [28], [49], [50] typically employ a strategy of rewriting dynamic link libraries by defining functions that share names with those in the CUDA runtime API and prioritizing the loading of these custom libraries by setting the *LD_PRELOAD* environment variable. This setup causes the dynamic linker to redirect calls from the original library functions to the custom library functions with the same name, effectively intercepting library functions. Subsequently, the custom library packages the function calls along with the required parameters and data, and sends them to the GPU server via RPC. It also modifies GPU memory management and the launching of CUDA kernel functions to ensure that RPCs are executed correctly on the GPU server. In this manner, these methods achieve transparent offloading by intercepting kernel functions at the system layer using similarly named functions in the dynamic link library and offloading their execution to the GPU server.

Compared to using the GPU on the mobile device, changes in the system call process primarily occur in step C. The detailed steps of the transparent offloading process (depicted in the right part in Fig. 2) are as follows:

- (1) The dynamic link library is modified such that each operator prioritizes calling the RPC functions that have the same names as those in the CUDA runtime API, enabling the effective identification and interception of all CUDA kernel function calls.
- (2) The transparent offloading client transmits the called CUDA runtime API and the required parameters to the GPU server via the MEC network using RPC.

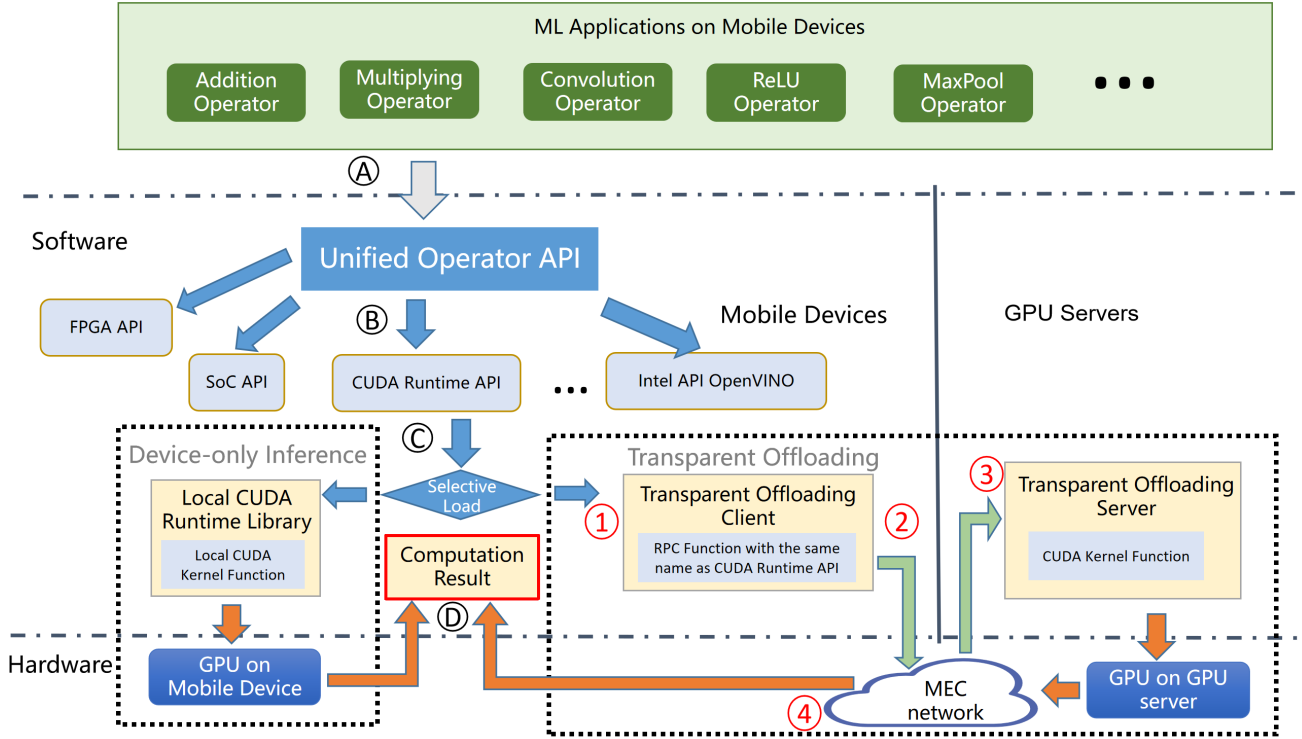
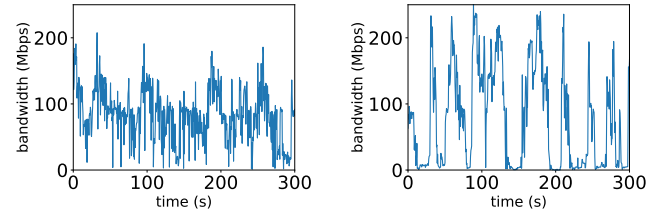


Fig. 2: Workflow of Transparent Offloading System for Model Inference in MEC environments.

- (3) The transparent offloading server on the GPU server launches the corresponding CUDA kernel functions and completes the computation.
- (4) The computation results are sent back to the client. The transparent offloading system then returns these results to the upper-layer function calls.



(a) Indoors

(b) Outdoors

Fig. 3: The wireless transmission instability of TCP between our robot and the base station in MEC networks.

D. Characteristics of MEC Networks

In real-world scenarios, mobile devices primarily rely on wireless networks, which offer high mobility but limited bandwidth capacity compared to data center networks equipped with high-speed technologies such as InfiniBand [51] or PCIe [52] (40–500 Gbps).

The bandwidth capacity of wireless networks is inherently constrained by both theoretical limits and practical challenges in MEC. While Wi-Fi 6 can achieve a peak bandwidth of 1.2 Gbps per stream [53], mobile devices lack the necessary hardware to fully leverage this capacity [54]. Furthermore, the actual available bandwidth varies significantly due to factors such as device mobility [55], [56], signal obstruction [57], [58], and channel contention [59], [60], further limiting effective bandwidth.

To examine wireless instability in MEC scenarios, we conducted a robot surveillance experiment where four-wheeled robots navigated through a lab (indoors) and a campus garden (outdoors) at speeds of 5–40 cm/s. Using iperf [61], we measured real-time wireless bandwidth capacity between the robot and a base station over TCP [62] at 0.1-second intervals for five minutes. As shown in Fig. 3, the average bandwidth was 93 Mbps indoors and 73 Mbps outdoors, with outdoor

measurements exhibiting higher fluctuations and occasional near-zero drops due to obstacles and reduced signal reflections. This indicates that transparent offloading is hard to sustain in real-world wireless networks, as the Round-Trip Time (RTT) for operator-level RPCs in traditional transparent offloading systems is usually higher than in data center networks.

while CUDA kernel launches are enqueued to GPU at the very beginning of the inference, and the GPU runs kernels in the queue while the CPU waits for all kernels to finish, the limited bandwidth in MEC makes the RTT time of each CUDA kernel launching prolonged (Sec.II-D).

RPC Optimization. Remote Procedure Call (RPC) [63] is a fundamental communication protocol enabling processes to request services from remote computers over a network. While common strategies like Caching [64] (storing results of previous RPC calls), Batching [65] (aggregating multiple RPC calls into a single request), and Asynchronous RPC [66] (allowing non-blocking execution so the client can perform other tasks while awaiting the server's response) aim to enhance

RPC performance, they prove largely ineffective in reducing the substantial communication costs of transparent offloading systems during model inference. Specifically, Caching fails because each inference typically processes unique input, necessitating fresh operator computations. Batching, while reducing the number of network requests by aggregating multiple calls, still cannot eliminate the per-operator RPC communication and can introduce latency as individual RPCs must wait to form a batch, particularly when the total number of RPCs for an inference is uncertain. Furthermore, Asynchronous RPC methods compromise the correctness of inference results because they cannot guarantee the sequential execution of GPU operations on the server. This sequential integrity is vital not only for launching CUDA kernels (“`cudaLaunchKernel`”) in the correct order but critically for “`cudaMemcpyHtoD`” and “`cudaMemcpyDtoH`” operations to manage input and output data accurately. These memory transfer operations, often involving larger data volumes and thus longer transmission times compared to kernel launches, are particularly prone to misordering under asynchronous execution, leading to corrupted data. In contrast, RRTO significantly reduces communication costs by eliminating most operator-level RPCs (Sec. III), representing a specialized co-design of RPC optimization and transparent offloading tailored for model inference.

III. OVERVIEW

A. Architecture of RRTO

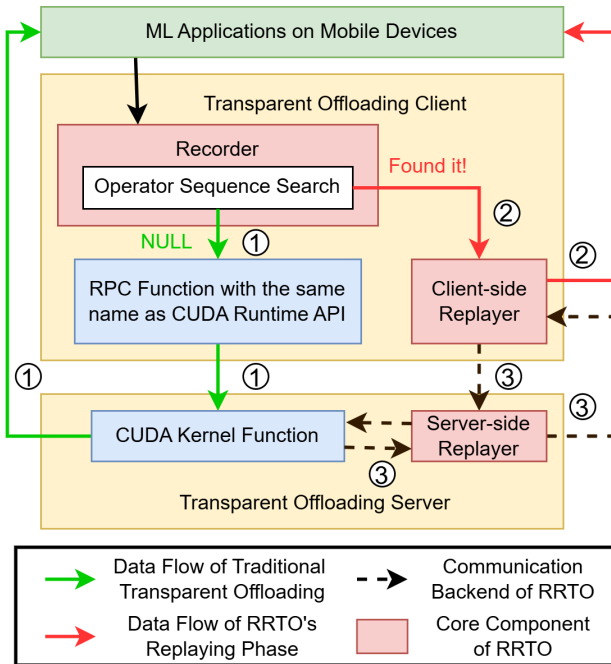


Fig. 4: Architecture of RRTO.

In this section, we present RRTO, a high-performance transparent offloading system for model inference in MEC, featuring a novel record/replay mechanism. Fig. 4 illustrates the architecture of RRTO. Compared to Fig. 2, RRTO integrates the record/replay mechanism into the core components of existing transparent offloading systems while maintaining transparency to upper-layer applications. This means RRTO

enables offloading without requiring any source code modifications. The pseudocode for both client and server sides is provided in Sec. IV-C.

During the first several inferences, RRTO enters the recording phase (①) and follows the execution pattern of traditional transparent offloading systems by offloading the execution of each operator to the GPU server via RPC, as illustrated by the green lines in Fig. 4. When intercepting CUDA kernel function calls from upper-layer ML applications, RRTO uses its recorder to log the invoked functions, including their parameters and return values. It then performs an operator sequence search to identify the inference operator sequence, as detailed in Sec. IV-B.

Once the recorder identifies the inference operator sequence, RRTO transitions to the replaying phase (②), where subsequent inferences are executed by replaying the sequence using the replayer on both the client and server, as shown by the red lines in Fig. 4. The client-side replayer returns the results of previous RPC calls to the upper-layer applications, functioning like RPC caching used in existing optimization methods described in Sec. VII. This setup allows the offloading client to continue invoking the system functions of subsequent operators until it reaches the ending operator, where it awaits the final computation result from the offloading server. At the same time, the server-side replayer performs the operator computations on the GPU server (③): replays the identified sequence, and sends the final result back to the client. This approach enables RRTO to achieve communication overhead close to that of non-transparent offloading by transmitting almost the same data (only the raw input and output) without per-operator RPC communication during offloading, as illustrated by the black dotted lines in Fig. 4.

When multiple ML applications run simultaneously on a mobile device, RRTO effectively distinguishes operators from different applications, improving performance for each. Existing transparent offloading systems already separate RPC functions from different ML applications, enabling shared use of remote GPUs across multiple applications and ensuring that results are accurately returned. Within a single inference process, RRTO uses its tailored record/replay mechanism to differentiate operators from distinct tasks. During the recording phase, its operator sequence search accurately identifies the correct sequence, which is especially useful for tasks using the same model within the same process (Sec. IV-B). In the replaying phase, the client-side replayer checks for failures in the predicted operator sequence and detects new inference tasks, triggering new inferences through RRTO's record/replay mechanism. However, to address performance issues caused by resource or network contention from concurrent tasks, multiple inference scheduling strategies are needed (Sec. VII), such as batching inference tasks, rather than depending solely on RRTO. Future work will focus on integrating these strategies into RRTO to further optimize performance.

B. Workflow of Replaying Phase

In this section, we present the workflow of RRTO during the replaying phase and explain how it eliminates the per-operator RPC communication required by existing transparent

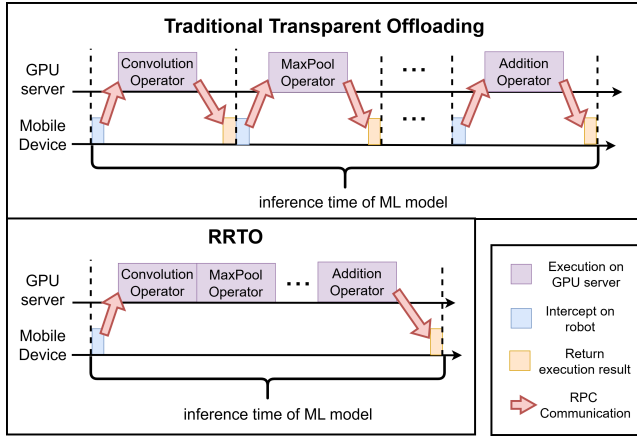


Fig. 5: Workflow of RRT0 during the replaying phase.

offloading methods via its record/replay mechanism. Fig. 5 illustrates this workflow and compares it with traditional transparent offloading systems during model inference in MEC networks. Traditional transparent offloading suffers from frequent operator-level RPC communication, leading to high communication overhead and degraded system performance (Sec. VI), including lower GPU utilization on servers, longer inference times, and increased energy consumption per inference.

To address the communication costs in the transparent offloading systems, RRT0 introduces an automatic recording and replay mechanism. Given that ML models in ML applications often exhibit static and predictable sequences of operator invocation during inference, RRT0 records the operators called during the initial inferences and replays this recorded sequence for subsequent inferences. As illustrated in Fig. 5, during the replay phase, RRT0 only offloads the first operator in the sequence via RPC as done in traditional transparent offloading systems, streamlining the start of the inference task. Subsequent operators are directly invoked on the GPU server side as in traditional systems. Consequently, RRT0 executes all required operators in the inference operator sequence with a single RPC at the beginning of each inference, eliminating the need for additional RPC communications for subsequent operators and significantly reducing communication costs.

IV. DESIGN

A. ML Models with Fixed-Order Operators

In machine learning, many models have a fixed order in activating their layers during the inference process, referred to as static ML models. These include: i) Feed-forward neural networks (e.g., Multi-Layer Perceptrons [67], Convolutional Neural Networks [68]) with a fixed structure where neurons in each layer are activated sequentially given an input; ii) Recurrent Neural Networks (e.g., simple RNNs [69], Elman networks [70]) that have a fixed computation process at each time step; iii) Autoencoders (e.g., basic autoencoders [71], Variational Autoencoders [72]) with fixed encoder and decoder parts that activate the same components during each inference; iv) Generative Adversarial Networks (e.g., basic

GANs [73], DCGANs [74]) with fixed generator and discriminator structures; v) shallow machine learning models (e.g., linear regression [75], logistic regression [76], Support Vector Machines [77]) that typically have a single fixed layer. These models with fixed structures and no dynamic mechanisms have relatively simple and regular computation processes and are the targeted models of our RRT0 and the baselines.

On the other hand, some models may activate different layers during different inferences depending on the input, and are referred to as dynamic ML models. These include i) Models with attention mechanisms (e.g., Transformers [78], BERT [79]) that dynamically compute attention weights to focus on different parts of the input; ii) Gated models (e.g., LSTM [80], GRU [81]) that control information flows through gating units, leading to different activated parts based on the input; iii) Conditional computation models (e.g., Mixture of Experts [82]) that select different experts to activate based on input conditions; iv) Dynamic network models (e.g., those obtained through Neural Architecture Search [83]) that can dynamically adjust their structure based on the input; v) Models with variable-length output sequences (e.g., GPT [84]), although the inference of each output unit (e.g., token) is static, the total number of iterations is input-dependent and unpredictable, as the final result is the entire output sequence. The dynamic nature of some ML models allows for better adaptability and expressiveness, but this nature also makes it difficult to profile their running statistics (time consumed, size of input and output, etc.) and thus few optimizing systems are targeted on these models.

Static ML models are widely used in mobile applications, such as Convolutional Neural Networks for computer vision tasks [9], [85], [86], and Multi-Layer Perceptrons, Recurrent Neural Networks, and Support Vector Machines for robot manipulation and autonomous navigation [87]–[89]. In contrast, dynamic ML models often demand greater computing resources and GPU storage, leading to their deployment primarily in data centers rather than on mobile devices [78], [79], [82]. The record/replay mechanism of RRT0 only fails when the operator sequence changes, which is typical in dynamic models. When the replayer on the mobile device detects inconsistencies between actual operator calls and the identified operator sequence, RRT0 temporarily disables the record/replay mechanism, falls back to traditional transparent offloading, and restarts the process until a new stable sequence is established. However, since ML models in mobile applications are typically static, such failures are rare, preventing performance degradation into traditional transparent methods. It is important to note that optimizing inference for dynamic ML models with changing operator sequences remains a challenge for all offloading systems, not just RRT0. Some studies [90], [91] address this issue by statistically predicting which layers will activate in future inferences, which falls outside the scope of this work, as we focus on static ML models. Some studies [90], [91] address this issue by statistically predicting which layers will activate in future inferences, which falls outside the scope of this paper, as we focus on static ML models.

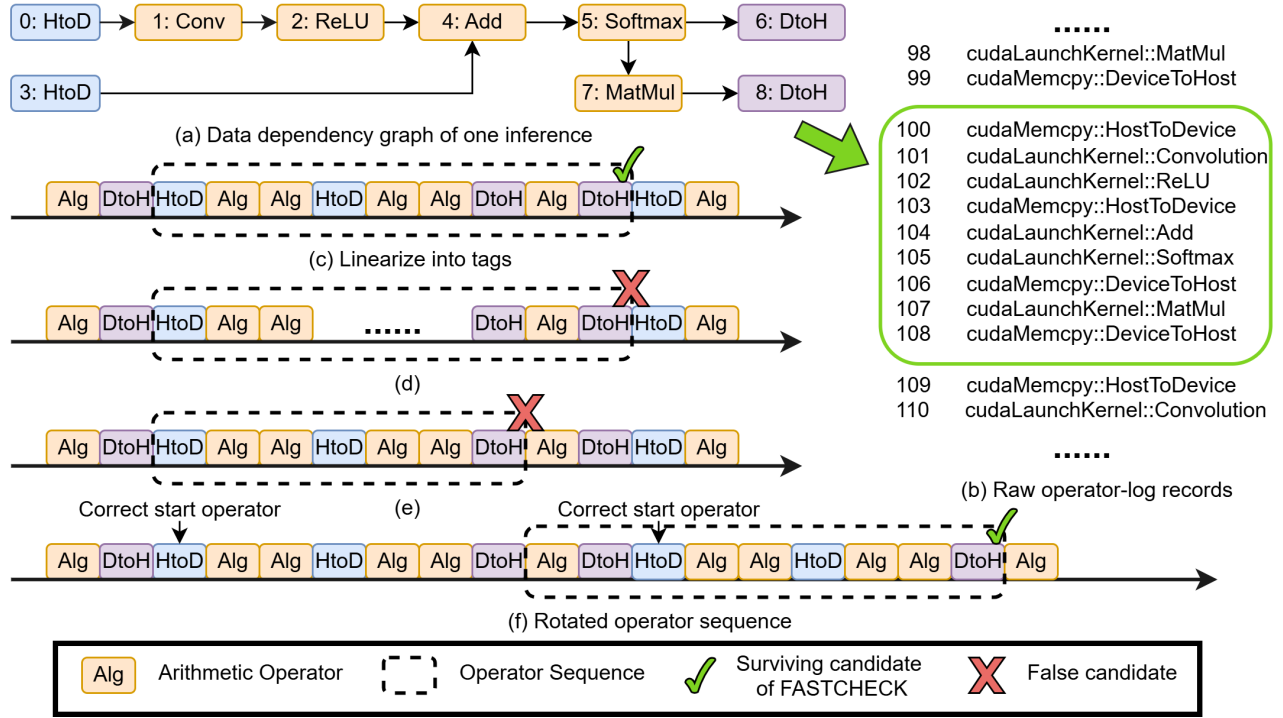


Fig. 6: Illustration of Operator Sequence Search.

B. Operator Sequence Search

The performance of RRTO hinges on its ability to accurately identify the correct inference operator sequence, since even a single missing or extra operator disrupts the end-to-end data flow and produces incorrect results. To achieve this under realistic conditions, Operator Sequence Search must therefore meet three fundamental challenges: i) Transparency: RRTO cannot rely on any high-level framework hooks or metadata and must infer the sequence solely from raw operator-log records. ii) Initialization variability: real ML models typically emit extra or altered operators during model loading and the very first inference; these one-time initialization artifacts must not be mistaken for the regular inference pattern. iii) Large trace size: real-world operator traces can contain tens or even hundreds of thousands of entries, so any brute-force search over all candidate subsequences would be prohibitively expensive. The pseudo code for the algorithm of operator sequence search is outlined in Alg. 1, and Fig. 6 gives an example of how these challenges are addressed in our algorithm.

To address these challenges, our algorithm exploits three key observations. ① A static ML model executes the same operator sequence for each inference, yielding a highly regular pattern in the logs. ② The real-time requirements of MEC enforces processing inference immediately, which begins with a host-to-device memory copy of the raw input (“cudaMemcpyHtoD”, short for “HtoD”) and ends with a device-to-host memory copy of the result (“cudaMemcpyDtoH”, short for “DtoH”). By treating these copies as special operators and grouping any following synchronization calls with them (e.g., “cudaStreamSynchronize”), we obtain reliable boundary markers. ③ The correct inference sequence must satisfy

all data-dependency constraints: every operator’s inputs must originate either from the raw input, from a prior operator’s output (share the same memory address), or from the model’s parameters. Unfortunately, relying solely on the maximum-repeated-substring algorithm [] based on observation ① fails to segment the operator sequence correctly, because when the sequence repeats continuously in the log, the algorithm merges multiple consecutive iterations into a single maximal substring (Fig. 6.d). Likewise, using only the memory-copy markers of observation ② falls short when multiple HtoD or DtoH events occur within a single inference, leaving ambiguous which copy corresponds to the true beginning or end (Fig. 6.e).

memory transfer operations

Building on these observations, we apply a two-stage matching strategy to shrink the search region and accelerate sequence identification. In the first stage, FASTCHECK scans the log to identify candidate start and end operators based on memory-copy boundaries of observation ② (lines 3 and 4) and then locates possible operator sequences by detecting repeated sequences based on observation ① (line 14). Each candidate ends at the current last end operator (line 10); if that operator truly marks the inference end, the candidate begins at the matching start operator (Fig. 6.c), whereas if it lies within a rotated sequence, the candidate instead begins at the next occurrence of an end operator (Fig. 6.f) (line 11). FASTCHECK then linearizes the log into a compact string of operator categories (“HtoD”, “DtoH”, arithmetic, etc.) that it can count in linear time how often the candidate substring reappears in the latter portion of the log (line 8) and thereby confirm sustained repetition despite initialization variability (line 27). By focusing solely on category tags and recurrence,

Algorithm 1 Operator_Sequence_Search

Input: Logs: list of OperatorInfo entries from the first N inferences; R : minimum number of repeats for check
Output: inference operator sequence IOS or NULL

```

1: function OPERATORSEQUENCESEARCH(Logs,  $R$ )
2:   // Phase 1: extract candidate start and end operators
3:    $S \leftarrow \{v.index \mid v.func = \text{"HtoD"}, v \in \text{Logs}\}$ 
4:    $T \leftarrow \{v.index \mid v.func = \text{"DtoH"}, v \in \text{Logs}\}$ 
5:   if  $S = \emptyset$  or  $T = \emptyset$  then return NULL
6:   end if
7:   // Phase 2: linearize into tags
8:    $\text{Tags} \leftarrow \{\text{Tag}(v) \mid v \in \text{Logs}\}$ 
9:   // Phase 3: search for a repeating sequence
10:   $end \leftarrow \text{MAX}(T)$ 
11:   $starts \leftarrow S \cup \{i + 1 \mid i \in T\}$ 
12:  for each  $j \in starts$  and  $j \leq end$  do
13:     $\ell \leftarrow end - j$ 
14:    if FASTCHECK(Tags,  $j, \ell, R$ ) then
15:      for each  $k \in S$  and  $j - \ell \leq k \leq j$  do
16:        if FULLCHECK(Logs,  $k, \ell, R, T$ ) then
17:           $\text{IOS} \leftarrow \text{Logs}[k \dots k + \ell - 1]$ 
18:          return IOS
19:        end if
20:      end for
21:    end if
22:  end for
23:  return NULL
24: end function
25: function FASTCHECK(Tags, start, length,  $R$ )
26:  count  $\leftarrow 0$ , pos  $\leftarrow$  start
27:  while pos  $\geq 0$  and Tags[start:start+length] =
    Tags[pos:pos+length] do
28:    count  $\leftarrow$  count + 1
29:    pos  $\leftarrow$  pos - length
30:  end while
31:  return count  $\geq R$ 
32: end function
33: function FULLCHECK(Logs, start, length,  $R, T$ )
34:  // Phase 1: check data dependency
35:  end  $\leftarrow$  start+length-1
36:  if end  $\notin T$  or CHECKDATADEPENDENCY(Logs, start, length) = False then
37:    return False
38:  end if
39:  // Phase 2: completeness check
40:  count  $\leftarrow 0$ , pos  $\leftarrow$  start
41:  while pos  $\geq 0$  do
42:    if for all  $0 \leq t < \text{length}$ , Logs[start+t]  $\equiv$  Logs[pos+t]
then
43:      count  $\leftarrow$  count + 1
44:      pos  $\leftarrow$  pos - length
45:    else
46:      break
47:    end if
48:  end while
49:  return count  $\geq R$ 
50: end function

```

FASTCHECK prunes the vast majority of false candidates caused by one-time initialization artifacts or spurious memory-copy boundaries.

Once FASTCHECK has identified the surviving candidates, operator sequence search advances to the second stage, FULLCHECK, which verifies each candidate exhaustively (line 16). Because a FASTCHECK candidate may be a cyclic rotation of the true sequence, FullCheck first realigns it to the correct start and end markers using the host-to-device and device-to-host boundaries from observation ② (Fig. 6.f, line 36). It then checks that every operator in the sequence satisfies the required data dependencies according to observation ③ (line 36) that inputs must come from raw input, a prior operator's output at the same address, or model parameters. Finally, it conducts a full one-to-one, record-level comparison across the entire log to confirm that the sequence repeats exactly as predicted by observation one (line 42). Although this final validation is the most time-consuming step, it is applied only to a small set of high-confidence candidates.

In this way, the operator sequence search algorithm addresses the three challenges, and locates the correct operator sequence efficiently bykeeping transparency through pure log analysis, discounting one-time initialization variability via repeated-sequence detection, and taming large traces with a two-stage FastCheck+FullCheck strategy. This design enables RRTO to extract the precise inference operator sequence without any framework support, even in demanding real-world deployments.

C. Record/Replay Mechanism

Here we describe how RRTO implements its record/replay mechanism. The transparent offloading process is outlined in two parts: the client component is detailed in Alg. 2, and the server component in Alg. 3.

In Alg. 2 on the offloading client side, RRTO takes a CUDA kernel function called by an operator and the requisite parameters as input. Initially, the algorithm checks whether the recorder has already identified the inference operator sequence to determine if the current phase should be recording or replaying. If the sequence has not been established, RRTO enters the recording phase (lines 5 to 9), which involves sending an RPC to the server (line 6), performing the operator sequence search (line 7), and capturing and recording the RPC execution result (line 8 to 9), adhering to the execution pattern of traditional transparent offloading systems. To enhance system efficiency, RRTO overlaps the operator sequence search with the RPC execution, allowing the algorithm to complete while the client awaits the RPC result. If the inference operator sequence is already identified, RRTO transitions to the replaying phase on the mobile device (line 11 to 23). This phase starts by initiating RRTOfor a new inference at the first operator (line 14), then returns the execution results of previous RPC calls for intermediate operators within the sequence (lines 24–25), synchronizes inference input at “cudaMemcpyHtoD” (line 18) and inference output at “cudaMemcpyDtoH” (line 21). Finally, it returns the execution result of each function to the upper-layer applications (line 28).

Algorithm 2 RRTO_on_Client

Input: Cuda kernel function called by the corresponding operator func and the required parameters args
Output: The execution result ret
Parameter: inference operator sequence IOS

```

1: IOS  $\leftarrow \emptyset$ , Logs  $\leftarrow \emptyset$ 
2: while True do
3:   // Keep serving Cuda kernel function func
4:   if IOS is NULL then
5:     // recorder
6:     SENDRPCSTO SERVER(func,args)
7:     IOS  $\leftarrow$  OPERATORSEQUENCESEARCH(Logs)
8:     ret  $\leftarrow$  GETRPCEXECUTIONRESULT()
9:     Logs  $\leftarrow$  Logs  $\cup \{(func,args,ret)\}$ 
10:  else
11:    // replayer on mobile device
12:    if func is IOS[0][“func”] then
13:      // start a new inference
14:      STARTRRTO(IOS)
15:    end if
16:    if func is “cudaMemcpyHtoD” then
17:      // transmit input args
18:      ret  $\leftarrow$  SENDRPCSTO SERVER(args)
19:    else if func is “cudaMemcpyDtoH” then
20:      // waiting for the final computation result
21:      ret  $\leftarrow$  WAITINGFORRRTO()
22:    else
23:      // returning the execution results of previous RPC calls
24:      idx  $\leftarrow$  FIND(IOS, func)
25:      ret  $\leftarrow$  IOS[idx][“ret”]
26:    end if
27:  end if
28:  RETURNRESULT(ret)
29: end while

```

Algorithm 3 RRTO_on_Server

Input: client task task
Parameter: inference operator sequence IOS, the execution result ret

```

1: IOS  $\leftarrow \emptyset$ 
2: while True do
3:   if task is SENDRPCSTO SERVER then
4:     // Same workflow as traditional transparent offloading systems
5:     func,args  $\leftarrow$  GETCLIENTINPUT()
6:     ret  $\leftarrow$  CUDARUNTIME LIBRARY(func,args)
7:   else if task is STARTRRTO then
8:     // replayer on server
9:     IOS  $\leftarrow$  GETCLIENTINPUT()
10:    for all Op  $\in$  IOS do
11:      args  $\leftarrow$  Op[“args”]
12:      if func is “cudaMemcpyHtoD” then
13:        input  $\leftarrow$  GETCLIENTINPUT()
14:        args  $\leftarrow$  RRTOFIXARGS(args,input)
15:      end if
16:      ret  $\leftarrow$  CUDARUNTIME LIBRARY(Op[“func”],args)
17:      if func is “cudaMemcpyDtoH” then
18:        SENDEXECUTIONRESULTBACK(ret)
19:      end if
20:    end for
21:  end if
22: end while

```

In Alg. 3, the RRTO offloading server continuously awaits tasks from the client, aligning its operational phase with that of the client to ensure synchronized processing. During the recording phase, the server processes RPC requests similarly to traditional transparent offloading systems (lines 4 to 6). As the client moves into the replaying phase on the mobile device, the offloading server concurrently transitions to its replaying phase (line 8 to 19), effectively replaying the execution of the recorded inference operator sequence. During this phase, RRTO configures each operator’s parameters (line 14), typically supplying the current inference input data or its memory addresses, to ensure accurate and efficient execution.

V. IMPLEMENTATION

In this section, we first elaborate on the implementation of RRTO, and we then introduce the experiment setup.

A. Implementing RRTO

Software. We implemented RRTO within Cricket’s code-base [28], a transparent offloading system that provides a virtualization layer for CUDA applications, enabling remote execution without the need for source code modifications and recompilation of applications. RRTO employs the same Remote Procedure Call (RPC) for communication operations as Cricket: Libtirpc [63], a transport-independent RPC library for Linux. We integrate RRTO’s recorder and replayer into the corresponding RPC functions in Cricket, allowing for seamless integration and efficient operation of the record/replay mechanism.

Hardware. The evaluation was conducted on a customized four-wheeled robot (Fig. 7), equipped with a Jetson Xavier NX [35] 8G onboard computer that is capable of CUDA-accelerated ML model inference. The system runs Ubuntu 20.04 with ROS Noetic and uses a dual-band USB network adapter (MediaTek MT76x2U) for wireless communication. Detailed hardware and sensor configurations are shown in Fig. 7.

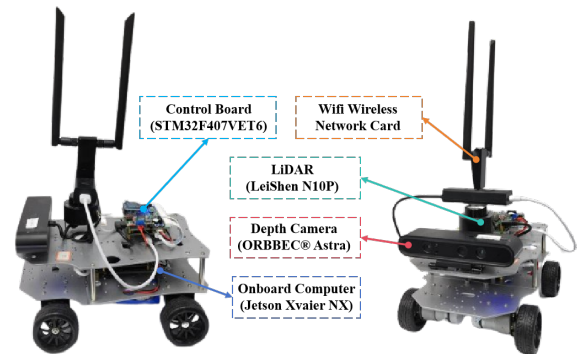


Fig. 7: The detailed composition of the robot platform.

TABLE II: Power draw (Watt) of our robot in different states.

	inference	communication	standby
Energy (Watt)	13.35	4.25	4.04

The GPU server is a PC equipped with an Intel(R) i5 12400f CPU @ 4.40GHz and an NVIDIA GeForce GTX 2080 Ti 11GB GPU, connected to our robot via Wi-Fi 6 over

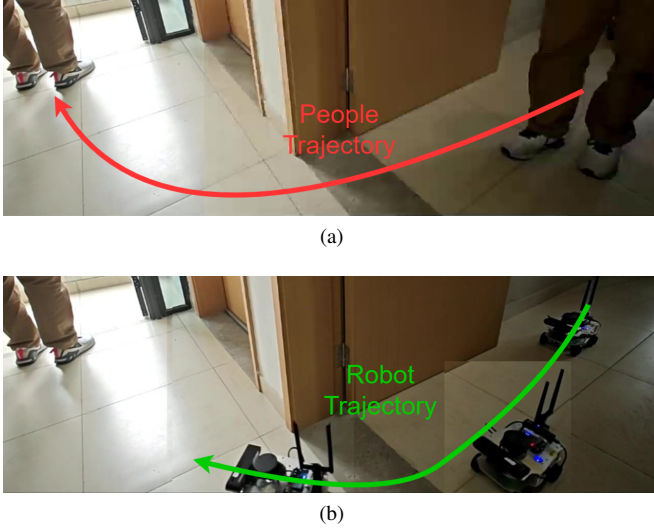


Fig. 8: Kapao [9], a real-time people-tracking application on our four-wheeled robot with a CNN-based human keypoint detection model.

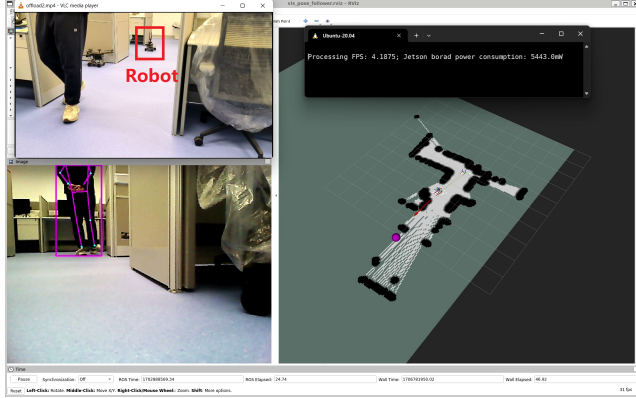


Fig. 9: A screenshot of our real-world experiment. The upper right corner displays real-time FPS and on-board energy consumption, the lower right corner shows the map created by the robot using its LiDAR, the lower left corner features the real-time view from the robot's camera, and the upper left corner provides a third-angle observation of the entire experimental process.

a 160MHz channel at 5GHz frequency. Tab. II summarizes the robots' on-board energy consumption (excluding motor power) in different states: inference (full GPU utilization, including CPU/GPU power), communication (communication with the GPU server, including wireless network card energy consumption), standby (no tasks to execute). Each Jetson Xavier NX is powered by a 21.6Wh battery, sustaining up to 1.6 hours of continuous model inference.

B. Experiment Setup

Task. We evaluated a real-time people-tracking robotic application on our robot as depicted in Fig. 8 and Fig. 9. The detailed workflow is described as follows: The ORBBEC Astra depth camera on our robot generates both RGB images and corresponding depth images. First, we obtain a person's key points in the RGB image using a well-known human pose

estimation model based on Convolutional Neural Networks, KAPO [9]. Then, by utilizing the depth values corresponding to these key points in the depth image, the points are mapped to a three-dimensional map constructed by the robot's LiDAR. A Kalman filter [92] is applied to filter out noise and obtain a more accurate position of the person. Finally, the STM32F407VET6 controller directs the robot to the target position, enabling real-time tracking of the person. KAPO continuously performs inference to achieve the fastest possible inference speed using both baselines and RRTO.

To verify the generalization of the performance of RRTO, we also evaluated six common robotic visual models on our robot useful for three categories of real-world mobile applications: i) ResNet [93] and ConvNext [94] for object classification; ii) FCN [95] and DeepLabv3 [96] for semantic segmentation; iii) FasterRCNN [97] and RetainNet [98] for object detection. We use the dataset of ImageNet [99] as their inference input and use their implementation from Torchvision [100].

Emulation Environments. We evaluated two real-world environments: indoors (robots move in our laboratory with desks and separators interfering with wireless signals) and outdoors (robots move in our campus garden with trees and bushes interfering with wireless signals, resulting in lower bandwidth). The corresponding bandwidths between the robot and the GPU server in indoors and outdoors scenarios are shown in Fig. 3.

Baselines. To comprehensively evaluate the performance of RRTO, we conducted comparative experiments against several baseline approaches:

- **Device-only inference ("Device-only"):** A conventional setup where the entire model is deployed and executed on the robot.
- **Native non-transparent offloading ("NNTTO"):** A non-transparent offloading method that deploys the entire model on a GPU server by modifying the source code. We evaluate NNTTO independently to demonstrate the benefits of offloading in inference latency and energy efficiency, and we use it as the theoretical upper bound for offloading approaches because it incurs minimal system transmission overhead by transmitting only inference inputs and outputs. This comparison underscores the communication time savings and performance gains achieved by RRTO.
- **Cricket [28]:** A state-of-the-art transparent offloading system designed for remote GPU usage.

Although advanced scheduling optimizations (e.g., layer partitioning [40]–[42] and multiple inference scheduling [43]–[48]) proven effective in non-transparent offloading systems can be adapted to RRTO, as discussed in Sec. VII, we have not yet incorporated these methods into RRTO and have restricted our comparisons to NNTTO. This is because the performance gains from these optimizations are orthogonal to our innovations, and such optimizations are typically not implemented in transparent offloading systems due to their poor performance associated with the significant communication cost. Consequently, we have excluded these optimizations

TABLE III: Inference time (Second), power draw (Watt) and energy consumption per inference (Joule) with standard deviation ($\pm n$) of Kapao in different environments with different systems.

Model(number of parameters)	System	Inference time (s)		power draw (W)		Energy consumption per inference (J)	
		indoors	outdoors	indoors	outdoors	indoors	outdoors
Kapao(77M)	Local	1.42(± 0.04)	1.42(± 0.04)	10.02(± 0.46)	10.02(± 0.46)	14.26(± 0.4)	14.26(± 0.4)
	Cricket	7.37(± 0.42)	7.39(± 0.33)	4.84(± 0.27)	5.05(± 0.28)	35.68(± 1.95)	37.3(± 2.06)
	NNTO	0.38(± 0.18)	0.42(± 0.16)	5.07(± 0.26)	4.89(± 0.09)	1.92(± 0.1)	2.07(± 0.04)
	RRTO	0.40(± 0.20)	0.44(± 0.19)	5.47(± 0.37)	5.27(± 0.41)	2.17(± 0.15)	2.34(± 0.18)

from our evaluation to ensure a fair comparison and plan to explore their integration into RRTO as part of our future work.

VI. EVALUATION

In this section, we evaluate the performance of RRTO from four aspects: i) comparison with baseline systems on inference latency and energy consumption in real-world mobile application; ii) sensitivity of RRTO in various MEC environments; iii) analysis of RRTO's record-replay mechanism iv) performance evaluation of RRTO on large-scale models common to fundamental mobile applications.

A. Superiority of RRTO

Our evaluation results of our robotic application, KAPAO, as presented in Tab. III, demonstrate that RRTO achieved performance comparable to non-transparent offloading system (NNTO) in terms of both inference time and energy consumption.

In terms of inference time, RRTO reduced inference time by an average of 72% compared to local computation and 95% compared to Cricket in the indoors scenario; the reductions in the outdoors scenario were 69% and 94%, respectively. Local computation, which processes the entire model on the robot without any data transmission to a GPU server, exhibits consistent performance in both indoor and outdoor scenarios. In contrast, the substantial communication costs associated with Cricket's frequent RPCs in its transparent offloading mechanism considerably slowed its inference times, a point that will be elaborated on in Sec. VI-B. By implementing its innovative record/replay mechanism, RRTO effectively minimized these extensive communication costs, achieving inference times comparable to those of NNTO, with nearly identical communication expenses.

In terms of energy consumption, RRTO achieved substantial reductions, decreasing energy usage per inference by an average of 85% compared to local computation and 94% compared to Cricket in indoor scenarios; outdoor reductions were 84% and 93%, respectively. Due to the intensive computational demands of the model, local computation incurs high power draw, whereas other systems that offload computations to a GPU server exhibit reduced energy usage. Although RRTO only reduced power draw by 45% compared to local computation and even showed a 13% increase in power draw compared to Cricket, its shorter inference times resulted in significantly lower energy consumption per inference. It is important to note that the average power draw values listed in Table III do not correspond to those in Table II. This discrepancy

arises because our application does not fully utilize the GPU capabilities of the Jetson Xavier NX, resulting in lower average energy consumption during local computation than during the inference stage. Furthermore, additional CPU computing tasks, such as robot control, cause the average energy consumption of all offloading systems to increase above levels observed during communication and standby phases.

The prolonged inference times observed in outdoor scenarios for all offloading systems can be attributed to the lower bandwidth available outdoors (see Sec. ??), which results in extended transmission times compared to indoor scenarios. By analyzing performance in both indoor and outdoor settings, we found that RRTO is robust across various MEC environments. This robustness stems from RRTO's ability to eliminate the frequent transmission requirements of RPCs in Cricket, thereby reducing communication overhead to levels comparable to NNTO. Moreover, when GPU servers reside in commercial cloud environments, network congestion and routing inefficiencies further restrict available bandwidth [101] and drive up transmission costs, making RRTO even more advantageous than Cricket.

B. Micro-Event Analysis

To gain a deeper insight into the performance improvements facilitated by RRTO, we analyzed the RPC function calls made by Cricket during various stages of KAPAO inference, illustrating the characteristics of traditional transparent offloading mechanisms. A detailed breakdown of these calls is provided in Table IV.

By comparing the different stages of function calls in Table IV, we can see that KAPAO undergoes an initialization stage of inference different from subsequent inference loops. This is because the working process of KAPAO [9] follows the default detection model in Yolo v5 [102]: the inference pipeline is first initialized by generating a mesh grid of a certain size that fits the input image size, which serves as the storage of intermediates; then in the following loop iterations through the inference pipeline the mesh grid is reused and the operator call sequence is fixed. RRTO recorded all involved operators during the first few inferences, not just the initial process, and ignored the different operator sequences from the initializing inference when the correct operator sequence was found.

In the loop inference detailed in Table IV, we observed that a significant portion, specifically 90.62%, of RPC function calls consisted of `cudaGetDevice` and `cudaGetLastError`. These calls, generated by PyTorch [16] due to our application's reliance on this framework, are crucial for determining the

TABLE IV: Composition of RPC function calls during different stages of KAPAO inference.

CUDA Runtime API	Composition during loading model	Composition during initializing inference	Composition during the following inference loop
cudaGetDevice	46858 (82.32%)	4789 (80.12%)	4735 (80.32%)
cudaGetLastError	4244 (7.46%)	616 (10.31%)	607 (10.30%)
cudaLaunchKernel	2752 (4.83%)	523 (8.75%)	522 (8.85%)
cudaMalloc	65 (0.11%)	4 (0.07%)	0 (0.00%)
cudaStreamIsCapturing	68 (0.12%)	4 (0.07%)	0 (0.00%)
cudaStreamSynchronize	1118 (1.96%)	16 (0.27%)	11 (0.19%)
cudaMemcpyHtoD	1117 (1.96%)	7 (0.12%)	3 (0.05%)
cudaMemcpyDtoH	1 (0.002%)	9 (0.15%)	8 (0.14%)
cudaMemcpyDtoD	701 (1.23%)	9 (0.15%)	9 (0.15%)

TABLE V: Semi-RRTO: only applying Caching [64] specifically to the RPCs of cudaGetDevice and cudaGetLastError in RRTO, effectively eliminating their transmission requirements.

Model	System	Inference time (s)		power draw (W)		Energy consumption per inference (J)	
		indoors	outdoors	indoors	outdoors	indoors	outdoors
Kapao(77M)	Semi-RRTO	1.56(± 0.24)	1.99(± 0.26)	5.47(± 0.37)	5.27(± 0.41)	8.54(± 0.58)	10.5(± 0.81)

data's location, facilitating computations across multiple GPUs and parallel tasks. Despite restricting PyTorch to use only a single GPU sequentially and employing Caching (referred to as "semi-RRTO" in Tab. V) to reduce RPC functions that require one round-trip time (RTT) to access the GPU server, semi-RRTO achieved inference times comparable only to local computation in our experiments and did not reach the speeds observed with NNTO. This is evident from the fact that cudaLaunchKernel still represents 8.85% of total RPC function calls, which are essential for notifying the server about subsequent computing tasks like additional convolution or maxpool operations. While traditional RPC optimization methods depend on waiting for cudaLaunchKernel RPCs from the client to direct the server's subsequent computing tasks, RRTO records these cudaLaunchKernel function calls and directly executes the subsequent computing tasks on the server, thereby eliminating the need for ongoing communication with the client.

Regarding the remaining RPC functions, namely cudaMemcpy, cudaStreamIsCapturing, cudaStreamSynchronize, and cudaMemcpyDtoD, which collectively account for 0.34% of the total RPC calls, they primarily handle data transmission and synchronization within the GPU and can also be replayed by RRTO on the server. However, cudaMemcpyHtoD and cudaMemcpyDtoH, accounting for 0.19% of the total RPC calls, are primarily used for data transmission between the CPU and GPU, which are mainly employed for the input and output of the ML model and cannot be replayed by RRTO.

To further illustrate the performance gains achieved by RRTO, we compared it with baseline systems in terms of the number of RPC calls and the resulting average GPU utilization on the GPU server during robotic application execution, as measured using pynvml [103] and presented in Table VII. Unlike RRTO and Cricket, NNTO bypasses RPC by directly synchronizing input and output data of the ML model between the CPU and GPU via cudaMemcpyHtoD and cudaMemcpyDtoH at the application layer, requiring modifications to the source code. Cricket, on the other hand,

experiences higher communication costs, which contribute to lower GPU utilization on the GPU server. Although RRTO also manages cudaMemcpyDtoH and cudaMemcpyHtoD like Cricket, resulting in 11 RPCs per inference, the performance improvements offered by RRTO are clearly advantageous.

C. Validation on A Wider Range of Models

We conducted a comprehensive evaluation of RRTO and other baseline systems across a diverse set of models commonly used in mobile devices, varying in parameter counts as detailed in Table VI. We selected the two most prevalent models for each of the three fundamental robotic tasks (object classification, semantic segmentation, and object detection) to assess the generalizability of RRTO's performance. Our findings confirm that RRTO achieves performance comparable to NNTO without necessitating any modifications to the source code. Cricket sometimes exhibits slower performance indoors compared to outdoors, primarily due to its exceptionally prolonged inference times and the unstable network fluctuations as shown in Fig. 3. While RRTO consistently outperforms across various models, we noted that the performance gains are relatively smaller for models with fewer parameters. This observation can be attributed to the fact that models with larger computational demands benefit more significantly from the robust computing power of the GPU server. Additionally, the substantial energy consumption incurred by extensive computations on the robot suggests that models with a larger number of parameters are more suited for offloading, thereby deriving greater benefits from offloading.

VII. RELATED WORK AND DISCUSSION

Fixed Calculation Logic. RRTO leverages the characteristic that operators in the inference of a DNN model often follow a fixed order, allowing its record/replay mechanism to support other computational tasks [104], [105], not solely static ML models, provided they exhibit fixed computational logic. However, RRTO is unable to support tasks with unfixed computational logic, such as dynamic ML models with

TABLE VI: Inference time (Second), power draw (Watt), and energy consumption per inference (Joule) with standard deviation ($\pm n$) of torchvision models in different environments with different systems.

Model(number of parameters)	System	Inference time (s)		power draw (W)		Energy consumption per inference (J)	
		indoors	outdoors	indoors	outdoors	indoors	outdoors
ResNet101(44M)	Local	0.10(± 0.02)	0.10(± 0.02)	10.61(± 0.25)	10.61(± 0.25)	1.02(± 0.21)	1.02(± 0.21)
	Cricket	7.41(± 0.37)	7.86(± 0.70)	4.98(± 0.43)	4.74(± 0.23)	36.91(± 3.21)	37.26(± 1.83)
	NNTO	0.09(± 0.09)	0.09(± 0.11)	5.32(± 0.32)	5.16(± 0.3)	0.45(± 0.03)	0.47(± 0.03)
	RRTO	0.09(± 0.11)	0.10(± 0.11)	5.05(± 0.3)	5.16(± 0.37)	0.45(± 0.03)	0.5(± 0.04)
ConvNext(197M)	Local	0.34(± 0.02)	0.34(± 0.02)	10.92(± 0.45)	10.92(± 0.45)	3.69(± 0.24)	3.69(± 0.24)
	Cricket	4.53(± 0.33)	4.78(± 0.57)	4.85(± 0.22)	4.84(± 0.22)	21.96(± 0.98)	23.1(± 1.04)
	NNTO	0.41(± 0.15)	0.42(± 0.11)	4.94(± 0.27)	4.66(± 0.25)	2.04(± 0.11)	1.97(± 0.11)
	RRTO	0.43(± 0.17)	0.44(± 0.16)	5.11(± 0.3)	5.07(± 0.33)	2.22(± 0.13)	2.25(± 0.14)
FCN(35M)	Local	1.44(± 0.33)	1.44(± 0.33)	6.0(± 2.69)	6.0(± 2.69)	8.62(± 1.96)	8.62(± 1.96)
	Cricket	4.44(± 0.28)	4.46(± 0.33)	4.84(± 0.25)	4.73(± 0.25)	21.01(± 1.12)	21.56(± 1.13)
	NNTO	0.17(± 0.12)	0.26(± 2.17)	4.97(± 0.32)	4.91(± 0.41)	0.83(± 0.05)	1.27(± 0.11)
	RRTO	0.17(± 0.14)	0.28(± 2.45)	5.04(± 0.34)	5.0(± 0.31)	0.88(± 0.06)	1.4(± 0.09)
DeepLabv3(42M)	Local	1.65(± 0.49)	1.65(± 0.49)	6.01(± 2.53)	6.01(± 2.53)	9.93(± 2.96)	9.93(± 2.96)
	Cricket	4.87(± 0.31)	4.76(± 0.53)	4.8(± 0.23)	4.7(± 0.26)	22.38(± 1.11)	22.39(± 1.22)
	NNTO	0.17(± 0.10)	0.18(± 0.12)	5.03(± 0.27)	4.81(± 0.29)	0.86(± 0.05)	0.88(± 0.05)
	RRTO	0.18(± 0.13)	0.19(± 0.13)	5.0(± 0.32)	5.02(± 0.34)	0.9(± 0.06)	0.97(± 0.07)
FasterRCNN(43M)	Local	2.96(± 0.51)	2.96(± 0.51)	10.96(± 0.84)	10.96(± 0.84)	32.42(± 5.56)	32.42(± 5.56)
	Cricket	7.86(± 0.33)	7.91(± 1.25)	4.7(± 0.27)	4.62(± 0.28)	36.91(± 2.1)	36.57(± 2.19)
	NNTO	0.16(± 0.91)	0.18(± 1.90)	4.93(± 0.29)	4.65(± 0.22)	0.78(± 0.05)	0.85(± 0.04)
	RRTO	0.17(± 1.00)	0.20(± 2.33)	4.66(± 0.33)	5.4(± 0.8)	0.78(± 0.06)	1.06(± 0.16)
RetinaNet(38M)	Local	1.59(± 0.18)	1.59(± 0.18)	10.32(± 0.7)	10.32(± 0.7)	16.37(± 1.9)	16.37(± 1.9)
	Cricket	9.99(± 0.34)	9.81(± 0.42)	4.84(± 0.23)	4.7(± 0.25)	48.34(± 2.31)	46.15(± 2.46)
	NNTO	0.15(± 0.90)	0.21(± 4.06)	4.97(± 0.32)	4.67(± 0.19)	0.73(± 0.05)	0.99(± 0.04)
	RRTO	0.15(± 1.01)	0.23(± 4.73)	5.13(± 0.36)	5.04(± 0.35)	0.79(± 0.06)	1.17(± 0.08)

TABLE VII: Comparison between RRTO and the baselines about numbers of RPC calls and average GPU utilization on the edge server

	NNTO	Cricket	RRTO
RPCs for each inference	NA	5895	11
Average GPU utilization on the GPU server	29.0%	1.1%	27.5%

changing operator sequences or tasks involving complex logic and branching, due to its inability to replay varying operator sequences. Moreover, tasks that entail complex logic and branching are generally more suited for CPU rather than GPU execution [106], and optimizing inference for dynamic ML models with changing operator sequences remains a pervasive challenge across all offloading systems.

Layer partitioning. Layer partitioning [40]–[42] optimizes individual inference tasks by distributing layers of a ML model across mobile devices and GPU servers to enhance end-to-end inference speed and energy efficiency. These Layer partitioning methods adopt various layers partitioning scheduling strategies depend on multiple factors, including model architecture, hardware capabilities (e.g., GPU servers and mobile devices), network conditions, and application-specific trade-offs between inference speed and energy efficiency. As these methods only consider about the end-to-end performance and the transmission overhead of the transmission, they traditionally favored non-transparent offloading systems due to the communication overhead penalties in transparent offloading systems and the lack of the model architecture. if a transparent offloading system like RRTO achieves performance comparable to non-transparent systems, such Layer partitioning methods can be effectively integrated into a high-performance transparent offloading system, thus offering sophisticated scheduling within a transparent framework. As a

model layer often corresponds to one or several operators (e.g., the convolution layer invoking a convolution operator via calling the “cudaLaunchKernel” function to launch the corresponding CUDA kernel), these layer partitioning scheduling strategies can be applied into RRTO by utilizing the the data dependency of inference operator sequence in operator sequence search to obtain the model architecture and refining the scheduling granularity from layer to operator granularity.

Multiple inference. Multiple inference scheduling optimizes overall system efficiency by coordinating multiple DNN inference tasks at a granularity broader than NNTO. This approach employs diverse decision algorithms to strategically determine the execution location and timing for numerous inference tasks, utilizing techniques such as task batching [43], urgency-based prioritization [44], [45], and deep reinforcement learning controls [46]–[48]. Distinct from layer partitioning methods that optimize individual inferences, these strategies coordinate a holistic offloading plan across multiple tasks to collectively minimize overall inference latency and energy consumption. Consequently, by adopting the decision algorithms from established multiple inference scheduling frameworks, RRTO can efficiently manage simultaneous inference tasks, maintaining high performance as detailed in Sec. III-A. These advanced scheduling optimizations, typically developed for non-transparent offloading systems, can be readily adapted to RRTO. Specifically, for multiple inference scheduling, their decision algorithms can be seamlessly integrated into RRTO to determine each inference task’s execution location and start time during its replay process, mirroring their function in non-transparent systems. This integration is viable because these methods prioritize offloading system performance, irrespective of system transparency. Although historically such methods were predominantly applied in non-transparent systems be-

cause of the performance limitations imposed by communication costs in transparent systems, a transparent offloading system like RRTO, if capable of achieving comparable performance, can successfully implement these techniques to deliver a high-performance, transparent solution with sophisticated multi-task scheduling.

Model Compression. Quantization and model distillation are two most common model compression techniques for mobile devices. Quantization [107]–[109] reduces precision of model weights and activations to lower computation costs, while model distillation [110]–[112] trains a smaller model to mimic a larger one with fewer resources. Unlike model compression techniques that trade accuracy for efficiency, offloading methods are orthogonal, achieving fast inference without compromising accuracy by effectively scheduling computational tasks.

VIII. CONCLUSION

In this paper, we introduce RRTO, a high-performance transparent offloading system optimized for ML model inference on robotic IoT. RRTO addresses the substantial communication costs typically associated with frequent RPCs in traditional transparent offloading systems by implementing a novel record/replay mechanism. This approach achieves comparable high performance to existing non-transparent offloading methods without necessitating any modifications to the source code. We anticipate that RRTO will significantly enhance the deployment of various ML applications on mobile robots in real-world environments. By providing fast and energy-efficient inference capabilities, RRTO enables these robots to perform complex tasks with increased efficiency and effectiveness, while maintaining a user-friendly interface. We anticipate that RRTO will foster the deployment of diverse robotic tasks utilizing ML models on real-world robots in the field by providing fast, energy-efficient, and easy-to-use inference capabilities.

REFERENCES

- [1] N. Chawla and S. Dalal, "Edge ai with wearable iot: A review on leveraging edge intelligence in wearables for smart healthcare," *Green Internet of Things for Smart Cities*, pp. 205–231, 2021.
- [2] H. Nong, M. Jin, C. Pan, H. Zhou, C. Zhang, X. Pan, Y. Chen, X. Wei, Y. Lu, K. Zhao *et al.*, "Intelligent sensing technologies based on flexible wearable sensors: A review," *IEEE Sensors Journal*, 2024.
- [3] R. Ravindran, M. J. Santora, and M. M. Jamali, "Multi-object detection and tracking, based on dnn, for autonomous vehicles: A review," *IEEE Sensors Journal*, vol. 21, no. 5, pp. 5668–5677, 2020.
- [4] S. Almutairi and A. Barnawi, "Securing dnn for smart vehicles: An overview of adversarial attacks, defenses, and frameworks," *Journal of Engineering and Applied Science*, vol. 70, no. 1, p. 16, 2023.
- [5] J. Xu, M. Kovatsch, D. Mattern, F. Mazza, M. Harasic, A. Paschke, and S. Lucia, "A review on ai for smart manufacturing: Deep learning challenges and solutions," *Applied Sciences*, vol. 12, no. 16, p. 8239, 2022.
- [6] P. Zheng, H. Wang, Z. Sang, R. Y. Zhong, Y. Liu, C. Liu, K. Mubarak, S. Yu, and X. Xu, "Smart manufacturing systems for industry 4.0: Conceptual framework, scenarios, and future perspectives," *Frontiers of Mechanical Engineering*, vol. 13, pp. 137–150, 2018.
- [7] K. J. Joseph, S. Khan, F. S. Khan, and V. N. Balasubramanian, "Towards open world object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 5830–5840.
- [8] S. Liu, X. Li, H. Lu, and Y. He, "Multi-object tracking meets moving uav," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2022, pp. 8876–8885.
- [9] W. McNally, K. Vats, A. Wong, and J. McPhee, "Rethinking keypoint representations: Modeling keypoints and poses as objects for multi-person human pose estimation," *arXiv preprint arXiv:2111.08557*, 2021.
- [10] Q. Li, F. Gama, A. Ribeiro, and A. Prorok, "Graph neural networks for decentralized multi-robot path planning," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 11 785–11 792.
- [11] J. Wang, Z. Sun, X. Guan, T. Shen, Z. Zhang, T. Duan, D. Huang, S. Zhao, and H. Cui, "Agnav: Efficient and energy-saving autonomous navigation for air-ground robots in occlusion-prone environments," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [12] Y. Yang, L. Juntao, and P. Lingling, "Multi-robot path planning based on a deep reinforcement learning dqn algorithm," *CAAI Transactions on Intelligence Technology*, vol. 5, no. 3, pp. 177–183, 2020.
- [13] A.-Q. Cao and R. de Charette, "Monoscene: Monocular 3d semantic scene completion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 3991–4001.
- [14] Y. Li, Z. Yu, C. Choy, C. Xiao, J. M. Alvarez, S. Fidler, C. Feng, and A. Anandkumar, "Voxformer: Sparse voxel transformer for camera-based 3d semantic scene completion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 9087–9098.
- [15] Z. Xia, Y. Liu, X. Li, X. Zhu, Y. Ma, Y. Li, Y. Hou, and Y. Qiao, "Scpnet: Semantic scene completion on point cloud," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 642–17 651.
- [16] pytorch, "pytorch," <https://www.pytorch.org>, 2024.
- [17] tensorflow, "tensorflow," <https://github.com/tensorflow/tensorflow>, 2024.
- [18] NVIDIA, "cuda runtime apis," <https://docs.nvidia.com/cuda/cuda-runtime-api/index.html>, 2024.
- [19] opencl, "opencl," <https://developer.nvidia.com/opencl>, 2024.
- [20] B. Plancher, S. M. Neuman, T. Bourgeat, S. Kuindersma, S. Devadas, and V. J. Reddi, "Accelerating robot dynamics gradients on a cpu, gpu, and fpga," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2335–2342, 2021.
- [21] T. Ohkawa, K. Yamashina, H. Kimura, K. Ootsu, and T. Yokota, "Fpga components for integrating fpgas into robot systems," *IEICE TRANSACTIONS on Information and Systems*, vol. 101, no. 2, pp. 363–375, 2018.
- [22] V. Honkote, D. Kurian, S. Muthukumar, D. Ghosh, S. Yada, K. Jain, B. Jackson, I. Klotchkov, M. R. Nimmagadda, S. Dattawadkar *et al.*, "2.4 a distributed autonomous and collaborative multi-robot system featuring a low-power robot soc in 22nm cmos for integrated battery-powered minibots," in *2019 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2019, pp. 48–50.
- [23] P. Davoodi, C. Gwon, G. Lai, and T. Morris, "Tensorrt inference with tensorflow," in *GPU Technology Conference*, 2019.
- [24] NVIDIA, "Nvidia cuda-x libraries," <https://developer.nvidia.com/gpu-accelerated-libraries>, 2024.
- [25] R. R. Fullerton and C. S. McWatters, "The production performance benefits from jit implementation," *Journal of operations management*, vol. 19, no. 1, pp. 81–96, 2001.
- [26] NVIDIA, "Getting started with cuda graphs," <https://developer.nvidia.com/blog/cuda-graphs/>, 2024.
- [27] torch, "torchscript," <https://pytorch.org/docs/stable/jit.html>, 2024.
- [28] N. Eiling, J. Baude, S. Lankes, and A. Monti, "Cricket: A virtualization layer for distributed execution of cuda applications with checkpoint/restart support," *Concurrency and Computation: Practice and Experience*, vol. 34, no. 14, 2022.
- [29] M. Schneider, F. Haag, A. K. Khalil, and D. A. Breunig, "Evaluation of communication technologies for distributed industrial control systems: Concept and evaluation of 5g and wifi 6," *Procedia CIRP*, vol. 107, pp. 588–593, 2022, leading manufacturing systems transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2212827122003146>
- [30] A. Ghosh, S. Iyengar, S. Lee, A. Rathore, and V. N. Padmanabhan, "React: Streaming video analytics on the edge with asynchronous cloud support," in *Proceedings of the 8th ACM/IEEE Conference on Internet of Things Design and Implementation*, 2023, pp. 222–235.

- [31] N. Ghielmetti, V. Loncar, M. Pierini, M. Roed, S. Summers, T. Aarrestad, C. Petersson, H. Linander, J. Ngadiuba, K. Lin *et al.*, "Real-time semantic segmentation on fpgas for autonomous vehicles with hls4ml," *Machine Learning: Science and Technology*, vol. 3, no. 4, p. 045011, 2022.
- [32] MLPerf, "Mlperf," <https://mlcommons.org/working-groups/benchmarks/mobile/>, 2023.
- [33] J. Beuchert and A. Rogers, "Snappergps: Algorithms for energy-efficient low-cost location estimation using gnss signal snapshots," in *Proceedings of the 19th ACM Conference on Embedded Networked Sensor Systems*, ser. SenSys '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 165–177. [Online]. Available: <https://doi-org.eproxy.lib.hku.hk/10.1145/3485730.3485931>
- [34] R. Pi, "Raspberry pi," <https://www.raspberrypi.com/documentation/computers/configuration.html#power-consumption>, 2022.
- [35] NVIDIA, "The world's smallest ai supercomputer," <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-xavier-series/>, 2024.
- [36] Z. Ning, M. Vandersteegen, K. Van Beeck, T. Goedemé, and P. Vandewalle, "Power consumption benchmark for embedded ai inference," 10 2024.
- [37] AnandTech, "Anandtech," <https://www.anandtech.com/show/17587/iphone-14-battery-life-test/4>, 2023.
- [38] C. Fernandez-Lozano, J. A. Seoane, M. Gestal, T. R. Gaunt, J. Dorado, and C. Campbell, "Texture classification using feature selection and kernel-based techniques," *Soft Computing*, vol. 19, no. 9, pp. 2469–2480, 2015.
- [39] W. Du, Z. Cao, T. Song, Y. Li, and Y. Liang, "A feature selection method based on multiple kernel learning with expression profiles of different types," *BioData mining*, vol. 10, pp. 1–16, 2017.
- [40] H. Wu, W. J. Knottenbelt, and K. Wolter, "An efficient application partitioning algorithm in mobile environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 7, pp. 1464–1480, 2019.
- [41] X. Chen, J. Zhang, B. Lin, Z. Chen, K. Wolter, and G. Min, "Energy-efficient offloading for dnn-based smart iot systems in cloud-edge environments," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 3, pp. 683–697, 2021.
- [42] H. Liang, Q. Sang, C. Hu, D. Cheng, X. Zhou, D. Wang, W. Bao, and Y. Wang, "Dnn surgery: Accelerating dnn inference on the edge through layer partitioning," *IEEE transactions on Cloud Computing*, 2023.
- [43] D. Zhang, N. Vance, Y. Zhang, M. T. Rashid, and D. Wang, "Edgebatch: Towards ai-empowered optimal task batching in intelligent edge systems," in *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2019, pp. 366–379.
- [44] L. Lin, X. Liao, H. Jin, and P. Li, "Computation offloading toward edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1584–1607, 2019.
- [45] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE communications surveys & tutorials*, vol. 19, no. 3, pp. 1628–1656, 2017.
- [46] M. Altamimi, A. Abdrabou, K. Naik, and A. Nayak, "Energy cost models of smartphones for task offloading to the cloud," *IEEE Transactions on Emerging Topics in Computing*, vol. 3, no. 3, pp. 384–398, 2015.
- [47] K. Elgazzar, P. Martin, and H. S. Hassanein, "Cloud-assisted computation offloading to support mobile services," *IEEE Transactions on Cloud Computing*, vol. 4, no. 3, pp. 279–292, 2014.
- [48] Z. Fang, T. Yu, O. J. Mengshoel, and R. K. Gupta, "Qos-aware scheduling of heterogeneous servers for inference in deep neural networks," in *Proceedings of the 2017 ACM Conference on Information and Knowledge Management*, 2017, pp. 2067–2070.
- [49] G. Giunta, R. Montella, G. Agrillo, and G. Coviello, "A gpgpu transparent virtualization component for high performance computing clouds," in *Euro-Par 2010-Parallel Processing: 16th International Euro-Par Conference, Ischia, Italy, August 31-September 3, 2010, Proceedings, Part I 16*. Springer, 2010, pp. 379–391.
- [50] Y.-S. Lin, C.-Y. Lin, C.-R. Lee, and Y.-C. Chung, "qcuda: Gpgpu virtualization for high bandwidth efficiency," in *2019 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 2019, pp. 95–102.
- [51] NVIDIA, "Infiniband networking solutions," <https://www.nvidia.com/en-us/networking/products/infiniband/>, 2024.
- [52] A. Li, S. L. Song, J. Chen, J. Li, X. Liu, N. R. Tallent, and K. J. Barker, "Evaluating modern gpu interconnect: Pcie, nvlink, nv-sli, nvswitch and gpudirect," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 94–110, 2019.
- [53] R. Liu and N. Choi, "A first look at wi-fi 6 in action: Throughput, latency, energy efficiency, and security," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 1, pp. 1–25, 2023.
- [54] X. Yang, H. Lin, Z. Li, F. Qian, X. Li, Z. He, X. Wu, X. Wang, Y. Liu, Z. Liao *et al.*, "Mobile access bandwidth in practice: Measurement, analysis, and implications," in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022, pp. 114–128.
- [55] A. Masiukiewicz, "Throughput comparison between the new hew 802.11 ax standard and 802.11 n/ac standards in selected distance windows," *International Journal of Electronics and Telecommunications*, vol. 65, no. 1, pp. 79–84, 2019.
- [56] Y. Pei, M. W. Mutka, and N. Xi, "Connectivity and bandwidth-aware real-time exploration in mobile robot networks," *Wireless Communications and Mobile Computing*, vol. 13, no. 9, pp. 847–863, 2013.
- [57] M. Ding, P. Wang, D. López-Pérez, G. Mao, and Z. Lin, "Performance impact of los and nlos transmissions in dense cellular networks," *IEEE Transactions on Wireless Communications*, vol. 15, no. 3, pp. 2365–2380, 2015.
- [58] N. I. Sarkar and O. Mussa, "The effect of people movement on wi-fi link throughput in indoor propagation environments," in *IEEE 2013 Tencon-Spring*. IEEE, 2013, pp. 562–566.
- [59] T. Adame, M. Carrascosa-Zamacois, and B. Bellalta, "Time-sensitive networking in ieee 802.11 be: On the way to low-latency wifi 7," *Sensors*, vol. 21, no. 15, p. 4954, 2021.
- [60] Y. Ren, C.-W. Tung, J.-C. Chen, and F. Y. Li, "Proportional and preemption-enabled traffic offloading for ip flow mobility: Algorithms and performance evaluation," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 12, pp. 12 095–12 108, 2018.
- [61] "iPerf - Download iPerf3 and original iPerf pre-compiled binaries." [Online]. Available: <https://iperf.fr/iperf-download.php>
- [62] Y. Tian, K. Xu, and N. Ansari, "Tcp in wireless environments: problems and solutions," *IEEE Communications Magazine*, vol. 43, no. 3, pp. S27–S32, 2005.
- [63] S. Dickson, "libtirpc: Transport independent rpc library," <https://git.linux-nfs.org/?p=steved/libtirpc.git>, 2024, git repository.
- [64] A. Singhvi, A. Akella, M. Anderson, R. Cauble, H. Deshmukh, D. Gibson, M. M. Martin, A. Strominger, T. F. Wenisch, and A. Vahdat, "Cliquesmap: Productionizing an rma-based distributed caching system," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, 2021, pp. 93–105.
- [65] N. Lazarev, S. Xiang, N. Adit, Z. Zhang, and C. Delimitrou, "Dagger: efficient and fast rpcs in cloud microservices with near-memory reconfigurable nics," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 36–51.
- [66] S. Eyerman and I. Hur, "Efficient asynchronous rpc calls for microservices: Deathstarbench study," *arXiv preprint arXiv:2209.13265*, 2022.
- [67] R. Kruse, S. Mostaghim, C. Borgelt, C. Braune, and M. Steinbrecher, "Multi-layer perceptrons," in *Computational intelligence: a methodological introduction*. Springer, 2022, pp. 53–124.
- [68] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: analysis, applications, and prospects," *IEEE transactions on neural networks and learning systems*, vol. 33, no. 12, pp. 6999–7019, 2021.
- [69] J. Koutnik, K. Greff, F. Gomez, and J. Schmidhuber, "A clockwork rnn," in *International conference on machine learning*. PMLR, 2014, pp. 1863–1871.
- [70] Y. Wang, L. Wang, F. Yang, W. Di, and Q. Chang, "Advantages of direct input-to-output connections in neural networks: The elman network for stock index forecasting," *Information Sciences*, vol. 547, pp. 1066–1079, 2021.
- [71] D. Meyer, "Introduction to autoencoders," 2015.
- [72] D. P. Kingma, M. Welling *et al.*, "An introduction to variational autoencoders," *Foundations and Trends® in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019.
- [73] Z. Pan, W. Yu, X. Yi, A. Khan, F. Yuan, and Y. Zheng, "Recent progress on generative adversarial networks (gans): A survey," *IEEE access*, vol. 7, pp. 36 322–36 333, 2019.
- [74] F. Gao, Y. Yang, J. Wang, J. Sun, E. Yang, and H. Zhou, "A deep convolutional generative adversarial networks (dcgans)-based semi-supervised method for object recognition in synthetic aperture radar (sar) images," *Remote Sensing*, vol. 10, no. 6, p. 846, 2018.
- [75] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to linear regression analysis*. John Wiley & Sons, 2021.
- [76] M. P. LaValley, "Logistic regression," *Circulation*, vol. 117, no. 18, pp. 2395–2399, 2008.

- [77] I. Steinwart and A. Christmann, *Support vector machines*. Springer Science & Business Media, 2008.
- [78] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [79] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [80] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: Lstm cells and network architectures," *Neural computation*, vol. 31, no. 7, pp. 1235–1270, 2019.
- [81] R. Dey and F. M. Salem, "Gate-variants of gated recurrent unit (gru) neural networks," in *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*. IEEE, 2017, pp. 1597–1600.
- [82] S. Masoudnia and R. Ebrahimpour, "Mixture of experts: a literature survey," *Artificial Intelligence Review*, vol. 42, pp. 275–293, 2014.
- [83] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, X. Chen, and X. Wang, "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–34, 2021.
- [84] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [85] L.-E. Montoya-Cavero, R. D. de León Torres, A. Gómez-Espinosa, and J. A. E. Cabello, "Vision systems for harvesting robots: Produce detection and localization," *Computers and electronics in agriculture*, vol. 192, p. 106562, 2022.
- [86] S. Wan and S. Goudos, "Faster r-cnn for multi-class fruit detection using a robotic vision system," *Computer Networks*, vol. 168, p. 107036, 2020.
- [87] S. M. J. Jalali, S. Ahmadian, A. Khosravi, S. Mirjalili, M. R. Mahmoudi, and S. Nahavandi, "Neuroevolution-based autonomous robot navigation: A comparative study," *Cognitive Systems Research*, vol. 62, pp. 35–43, 2020.
- [88] Z. Xie, L. Jin, X. Luo, Z. Sun, and M. Liu, "Rnn for repetitive motion generation of redundant robot manipulators: An orthogonal projection-based scheme," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 2, pp. 615–628, 2020.
- [89] F. Piltan, A. E. Prosvirnin, M. Sohaib, B. Saldivar, and J.-M. Kim, "An svm-based neural adaptive variable structure observer for fault diagnosis and fault-tolerant control of a robot manipulator," *Applied Sciences*, vol. 10, no. 4, p. 1344, 2020.
- [90] J. M. Tarnawski, A. Phanishayee, N. Devanur, D. Mahajan, and F. Nina Paravecino, "Efficient algorithms for device placement of dnn graph operators," *Advances in Neural Information Processing Systems*, vol. 33, pp. 15 451–15 463, 2020.
- [91] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (llms) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Transactions on Mobile Computing*, 2024.
- [92] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME-Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.
- [93] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [94] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," 2022. [Online]. Available: <https://arxiv.org/abs/2201.03545>
- [95] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," 2015. [Online]. Available: <https://arxiv.org/abs/1411.4038>
- [96] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017. [Online]. Available: <https://arxiv.org/abs/1706.05587>
- [97] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," 2016. [Online]. Available: <https://arxiv.org/abs/1506.01497>
- [98] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," 2018. [Online]. Available: <https://arxiv.org/abs/1708.02002>
- [99] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision (IJCV)*, vol. 115, no. 3, pp. 211–252, 2015.
- [100] S. Marcel and Y. Rodriguez, "Torchvision the machine-vision package of torch," in *Proceedings of the 18th ACM International Conference on Multimedia*, ser. MM '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 1485–1488. [Online]. Available: <https://doi.org/10.1145/1873951.1874254>
- [101] M. Noormohammadpour and C. S. Raghavendra, "Datacenter traffic control: Understanding techniques and tradeoffs," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 2, pp. 1492–1525, 2017.
- [102] G. Jocher, A. Chaurasia, A. Stoken, J. Borovec, NanoCode012, Y. Kwon, K. Michael, TaoXie, J. Fang, imyhxy, Lorna, Z. Yifu, C. Wong, A. V. D. Montes, Z. Wang, C. Fati, J. Nadar, Laughing, UnglvKitDe, V. Sonck, tkianai, yxNONG, P. Skalski, A. Hogan, D. Nair, M. Strobel, and M. Jain, "ultralytics/yolov5: v7.0 - YOLOv5 SOTA Realtime Instance Segmentation," Nov. 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.7347926>
- [103] N. Corporation, "pynvml: Python Bindings for the NVIDIA Management Library." [Online]. Available: <http://www.nvidia.com/>
- [104] N. Cautaeerts, P. Crout, H. W. Ánes, E. Prestat, J. Jeong, G. Dehm, and C. H. Liebscher, "Free, flexible and fast: Orientation mapping using the multi-core and gpu-accelerated template matching capabilities in the python-based open source 4d-stem analysis toolbox pyxem," *Ultramicroscopy*, vol. 237, p. 113517, 2022.
- [105] R. Chowdhury and D. Subramani, "Optimal path planning of autonomous marine vehicles in stochastic dynamic ocean flows using a gpu-accelerated algorithm," *IEEE Journal of Oceanic Engineering*, vol. 47, no. 4, pp. 864–879, 2022.
- [106] V. Rosenfeld, S. Breß, and V. Markl, "Query processing on heterogeneous cpu/gpu systems," *ACM Computing Surveys (CSUR)*, vol. 55, no. 1, pp. 1–38, 2022.
- [107] A. Défossez, Y. Adi, and G. Synnaeve, "Differentiable model compression via pseudo quantization noise," *arXiv preprint arXiv:2104.09987*, 2021.
- [108] t. Gheorghe and M. Ivanovici, "Model-based weight quantization for convolutional neural network compression," in *2021 16th International Conference on Engineering of Modern Electric Systems (EMES)*. IEEE, 2021, pp. 1–4.
- [109] C. Gong, Y. Chen, Y. Lu, T. Li, C. Hao, and D. Chen, "Vecq: Minimal loss dnn model compression with vectorized weight quantization," *IEEE Transactions on Computers*, vol. 70, no. 5, pp. 696–710, 2020.
- [110] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *International Journal of Computer Vision*, vol. 129, pp. 1789–1819, 2021.
- [111] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 2351–2363, 2020.
- [112] L. Wang and K.-J. Yoon, "Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 6, pp. 3048–3068, 2021.



Zekai Sun is currently working toward a PhD degree in the Systems and Networks Laboratory at the University of Hong Kong under the guidance of Prof. Heming Cui. He received Bachelor's Degree at University of Science and Technology of China (USTC) in 2020. His primary research areas include distributed systems, edge computing and systems for machine learning.



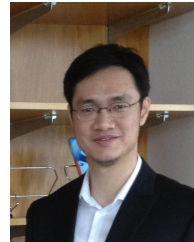
Xiuxian Guan is currently a HKU-SusTech Joint PhD student (2021Fall–Now) in Department of Computer Science, HKU and Department of Electrical & Electronic Engineering, SusTech. He received Bachelor's Degree in Department of Computer Science and Technology in University of Science and Technology of China (USTC) in 2020. His research interest includes distributed systems, edge computing and systems for machine learning.



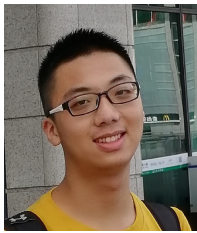
Fangming Liu received the B.Eng. degree from the Tsinghua University, Beijing, and the Ph.D. degree from the Hong Kong University of Science and Technology, Hong Kong. He is currently a Full Professor at the Huazhong University of Science and Technology, Wuhan, China. His research interests include cloud computing and edge computing, data center and green computing, SDN/NFV/5G, and applied ML/AI. He received the National Natural Science Fund (NSFC) for Excellent Young Scholars and the National Program Special Support for Top-Notch Young Professionals. He is a recipient of the Best Paper Award of IEEE/ACM IWQoS 2019, ACM e-Energy 2018 and IEEE GLOBECOM 2011, the First Class Prize of Natural Science of the Ministry of Education in China, as well as the Second Class Prize of National Natural Science Award in China.



Junming Wang is currently pursuing his MPhil in the Systems and Networks Laboratory at the University of Hong Kong under the guidance of Prof. Heming Cui. He received his Bachelor's degree at Lanzhou Jiaotong University. His primary research areas include embodied intelligence, autonomous navigation, and 3D computer vision.



Heming Cui is an Associate Professor in Computer Science of HKU. His research interests include operating systems, programming languages, distributed systems, and cloud computing, with a particular focus on building software infrastructures and tools to improve reliability and security of real-world software. He is a member of IEEE.



Yuhao Qing is currently a Ph.D. student in the department of computer science at the University of Hong Kong, under the supervision of Prof. Heming Cui. He received his bachelor's degree from the City University of Hong Kong. His research interests include machine learning systems and cloud computing.



Haoze Song is a Ph.D. student in the department of computer science at the University of Hong Kong. He received his bachelor's degree in Computer Science from the University of Science and Technology of China with honors. His current research interests are in the areas of distributed computing, data management in cloud computing environments, and scalable real-time systems.



Dong Huang is a Ph.D. student in the department of computer science at the University of Hong Kong. He received his bachelor's degree in Material Processing and Control Engineering from the Huazhong University of Science and Technology with honors. His current research interests are broadly in code generation such as correctness, efficiency, and fairness.